ADS Assignment Report - Ishan Batra - CSE 3 - 240485

Part 1: Avazu Click Rate Prediction

How often ads get clicked	How well the model works	What to do
About 17 of every 100 ads shown get clicked.	The 10% of ads it rated highest were clicked 41% of the time. The lowest 10% were clicked 1.8% of the time.	Spend more on highly rated ads, and test any big change on a small budget first.

1. Business Understanding

An ad platform shows ads on websites and inside apps. We assume it gets paid when an ad is clicked, so it wants to know which ads are likely to be clicked and put its budget there.

What we predict: the column *click*, which is 1 if an ad was clicked and 0 if not. The share of ads that get clicked is called the **click rate** (CTR).

Questions we asked: What share of ads get clicked? Which devices, apps, websites, positions and hours get more clicks? Can a model predict clicks better than a simple guess? Where should the budget go?

2. Dataset Summary and EDA

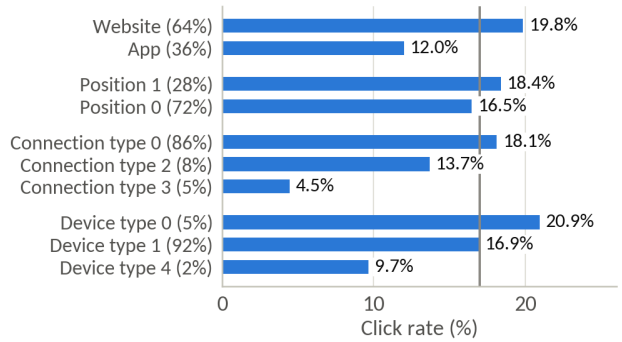
The data: click logs from Avazu's Kaggle competition: 40.4 million ads shown over 10 days in October 2014, already split into training (80%), validation (10%) and test (10%) rows. The dfSummary overview is in the notebook *01_eda*.

Our sample: every 40th row of each split, about 1 million rows, because that is what fits in our graphics card's memory. All 40 million rows come later, with batching (training in small chunks) and a neural network written from scratch. The sample still looks like the full data: the quietest hour has 11,877 rows in full and 298 in the sample, the busiest has 358,731 and 8,967, and hourly click rates differ only as much as chance would.

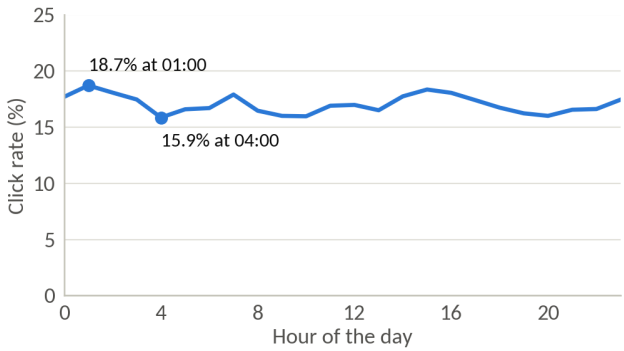
Columns	What they hold	Why they matter
click	Clicked or not (1 or 0)	This is what we predict
hour	Date and hour, stored as a number	People may click more at some times
banner_pos	Position of the ad, as a code	Some positions get more attention
site_id, site_domain, site_category	Which website, as scrambled text codes	Some websites get far more clicks
app_id, app_domain, app_category	Which app, as scrambled text codes	Some apps get far more clicks
device_id, device_ip, device_model, device_type, device_conn_type	Which device, its model and type, and its connection, as codes	They go along with clicks. The ID and IP point to individual people
C1, C14 to C21	Hidden columns, their meaning is not disclosed	They still help predict clicks

- **Size:** 24 columns, all of them categories, even the ones that look like numbers.
- **Missing values:** no blank cells, but some values stand in for "unknown". One device ID sits on 82.5% of rows, with 377,512 different IP addresses, which no single device could have. App ads carry filler values in the website columns and the reverse, so we read website categories on website ads only and app categories on app ads only. This is our reading of the patterns, since the data has no definitions.
- **Clicks and no clicks:** 17% clicked and 83% did not. A model that always says "no click" would be 83% correct and still useless, so we do not judge models by accuracy.

Some placements get far more clicks
In brackets: share of all ads. Grey line: all ads, 17.0%



The hour barely changes the click rate
It stays between 16% and 19% all day



- Website ads are clicked more than app ads: 19.8% against 12.0%.
- Position matters a little: position 1 gets 18.4% and position 0 gets 16.5%. Connection type matters more: type 0 gets 18.1%, types 3 and 5 under 5%.
- Device type 0 gets 20.9%, type 1 (92% of ads) 16.9%, types 4 and 5 about 9.5%. The data does not say what these codes mean, so we do not name them.
- Among the larger categories, click rates run from 28.3% down to 3.3% on websites and from 25.0% down to 9.0% in apps.
- The hour barely matters for click rate, even though traffic runs from 2% to 6% of ads per hour.

3. Machine Learning

Preparing the data:

- Split *hour* into hour of the day and day of the week.
- Removed the row number, and the device ID and IP address, because they identify individual people.
- Removed columns that add little, measuring what each one cost in log loss (explained below, lower is better). C14 saved 421 columns and cost 0.0009. C18, C19 and C21 saved 108 columns and cost 0.0001. app_domain saved 24 and cost 0.0001. C1 saved 5 and cost almost nothing. With all of them kept, the model does not fit in memory.
- Grouped very rare values (under 0.025% of training rows) into "other", then turned every category into yes or no columns, 1,356 in all.
- The test rows played no part in any of this, so nothing leaked from them into the model.

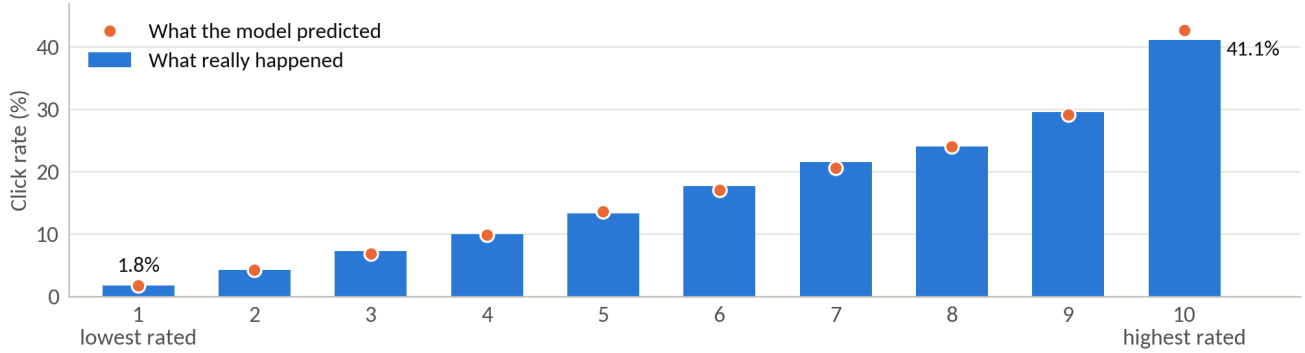
Model	In simple words	Log loss	ROC-AUC	PR-AUC	F1
Baseline	Always predicts the average click rate. Any useful model has to beat this.	0.458	0.500	0.171	0
Logistic regression	Gives each column a weight, adds them up and turns the total into a chance of a click.	**0.405**	**0.734**	**0.358**	**0.408**
Weighted logistic regression	The same, but clicks count about 5 times as much in training, so the rare clicks are not ignored.	0.603	0.734	0.357	0.407
Weighted linear SVM	Looks for a line that separates clicks from no clicks. It gives a score, not a chance, so it has no log loss.	-	0.717	0.312	0.396

- **Log loss:** how close the predicted chances were to what happened. Lower is better.
- **ROC-AUC:** take one clicked ad and one unclicked ad. This is how often the model rates the clicked one higher. 0.5 is guessing and 1 is perfect.
- **PR-AUC:** how many of the ads rated highest were really clicked. Guessing scores about 0.17, the share of clicks.
- **F1:** for a yes or no call, the balance between catching clicks and avoiding false alarms. We set the cut-off on the validation rows. The baseline never says yes, so it scores 0.

Scores are on 101,073 test rows. **Plain logistic regression is the best model:** its log loss is 11.5% lower than the baseline and it ranks ads better than the SVM. We wrote the models from scratch in PyTorch. Logistic regression matches the standard library (sklearn) to 3 decimals, and the SVM comes close (ROC-AUC 0.717 against 0.713).

4. Inference and Interpretation

Ads the model rates highly really do get clicked more



- **It ranks ads well.** The top 10% were clicked 41% of the time and the lowest 10% only 1.8%, a 23 times gap.

- **Its chances can be trusted.** The dots sit close to the bars. For the top group it says 43% and the real rate is 41%.
- **It is not just memorising.** Its log loss is about the same on rows it trained on (0.402) and rows it never saw (0.405).
- **Weighting does not help.** The weighted model ranks ads just as well, but its chances run far too high: 44% on average against a real 17%.
- **Most useful columns:** the ones that hurt most when removed are the app, C17 (a hidden column), the device model and the website.
- **Prediction is not cause.** The model shows what goes along with clicks. Position 1 gets more clicks, but better ads may simply be given position 1. Only an experiment that places ads at random (an A/B test) can show whether moving an ad changes its clicks.
- **Limits:** one row in 40 used; rare values grouped; hidden and scrambled columns that are hard to explain; only 10 days, with test rows from the same days, so new days will probably score a bit worse; a simple model that misses combinations (this app on this device); no history per user; no prices.

5. Recommendations

1. **Score every ad with plain logistic regression and bid by expected value:** chance of a click × price per click. The price is what the advertiser pays for each click. It is not in our data, so the platform has to supply it.
2. **Put the budget at the top of the list.** The top 30% of ads were clicked about 32% of the time, the bottom 30% about 4.5%.
3. **Test before shifting budget.** Website ads, connection type 0 and device type 0 look better than the rest. Connection types 3 and 5, device types 4 and 5, and the weakest website categories (f66779e6 and 76b2941d, under 4%) are the first places to cut. These are patterns and may not be causes, so try them on a small share of the budget in an A/B test.
4. **Do not bid with the weighted model,** because it overstates every chance.
5. **Next steps:** train on all 40 million rows, add a neural network that can learn combinations, add history per user, and check the model on the last day of data.

Part 2: Evaluating a Job Training Programme

Important: this data is synthetic, made for practice. It is not evidence about any real Indian government programme.

Effect of the offer	Who gains	Still open
At 30 months, being offered a place raised employment by 4.0 points and average earnings by ₹9,076.	Every social group gains by 30 months. Nothing shows that one group gains more than another.	We have not yet measured the effect of the training itself, and we have no cost data.

Business Understanding

A state skills mission offered subsidised training by lottery. Some of those offered never enrolled, and a few not offered enrolled anyway. Employment and earnings were recorded at 18 and 30 months. The government asks: what does offering training do, what does the training itself do, and do OBC, SC and ST applicants benefit? The file holds 40,000 applicants in 45 districts (the assignment says 10,000 in 16), and we used the file.

1. Data Understanding and Randomisation

- **The data:** 40,000 applicants and 25 columns. The only blanks are in training type, exactly for the 23,140 people who did not enrol, so they mean "no training".
- **Was the lottery fair?** We compared 14 background facts (past earnings, schooling, age, caste and so on) between the two groups. The largest gap is 0.025 standard deviations, which is tiny. One of the 14 tests came out below 5% (SC share, 18.3% against 17.4%), and chance alone would give about 0.7. Exactly 2 in 3 applicants were offered a place in every state. The lottery looks fair.

	Did not enrol	Enrolled	Total	Enrolment rate
Not offered	12,927	406	13,333	3.0%
Offered	10,213	16,454	26,667	61.7%

- **First stage:** the offer raised enrolment by 58.7 points (95% range 58.0 to 59.3).
- **Noncompliance:** 38% of those offered never enrolled, and 406 people enrolled without an offer. Comparing people who enrolled with people who did not would mix the effect of training with the kind of person who chooses to enrol.

2. Intent-to-Treat (ITT) Analysis

We compare everyone offered a place with everyone not offered, whether or not they enrolled. The lottery made the two groups alike, so the gap is caused by the offer.

Outcome	Offered	Not offered	Effect of the offer	95% range
Employed at 18 months	46.2%	43.1%	+3.1 points	2.1 to 4.1
Employed at 30 months	46.9%	43.0%	+4.0 points	2.9 to 5.0
Earnings at 18 months	₹58,712	₹52,161	+₹6,551	₹5,036 to ₹8,067
Earnings at 30 months	₹76,406	₹67,329	+₹9,076	₹7,084 to ₹11,069

- **How we test it:** Welch's t-test, which does not assume both groups vary equally (its standard errors are heteroskedasticity-robust). The standard error says how much the gap would move by chance from one sample to the next. The 95% range is the gap ± 1.96 standard errors. If we repeated the study 100 times, about 95 of the ranges would contain the true effect. Zero is far outside every range (all p values are below 0.001), so chance does not explain these gaps.
- **For policymakers:** on average, each applicant offered a place earns about ₹9,000 more at 30 months, 13.5% above those not offered, counting the 38% who never took the place up.
- **Does it grow over time?** Partly. The earnings effect rises by ₹2,525 from 18 to 30 months (95% range ₹212 to ₹4,838), though as a share of the not offered group's earnings it barely moves (12.6% to 13.5%). The rise in the employment effect (0.84 points) could be chance (p = 0.24). The file does not say what period earnings cover.
- **Not done yet:** repeating this while accounting for each person's background (baseline covariates), which the assignment asks for.

3. Instrumental Variables and LATE

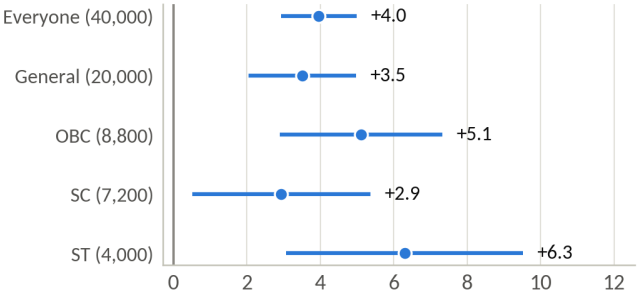
Not done yet. This step would estimate the effect of the training itself using the Wald estimator and two-stage least squares, and cover monotonicity, the exclusion restriction, compliers, always-takers and never-takers. Until then the government's second question stays open, and every effect in this report is the effect of the offer.

4. Earnings, Employment and Heterogeneous Effects

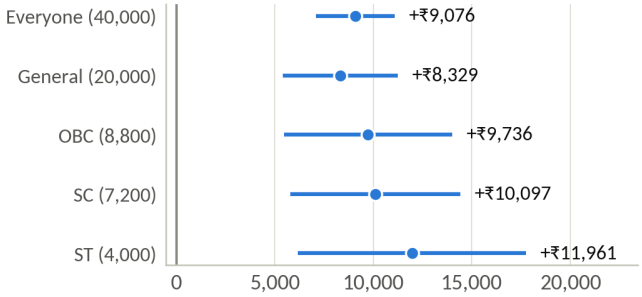
Zero earnings: at 30 months, 57.0% of those not offered and 53.1% of those offered earn nothing, and earnings are 0 exactly when someone is not employed. So the ₹9,076 has two parts: ₹6,201 from more people working and ₹2,875 from offered workers earning more (arithmetic only). Comparing the employed alone is not fair, because the offer changes who works: offered workers have 0.11 fewer years of schooling (p = 0.02), while the full groups match.

Heterogeneous effects means the effect may differ between kinds of people. We ran the same comparison inside each social category, which is fair because the lottery ran in every group and a person's category is fixed before the offer.

Employment at 30 months (points)



Earnings at 30 months (₹)



Each dot is the effect and each line is its 95% range. The number of people is in brackets.

Group	Employed, 18 months	Employed, 30 months	Earnings, 18 months	Earnings, 30 months
General	+3.6	+3.5	+₹6,687	+₹8,329
OBC	+2.4 *	+5.1	+₹6,871	+₹9,736
SC	+4.0	+2.9 *	+₹8,229	+₹10,097
ST	+1.5 *	+6.3	+₹4,091 *	+₹11,961

Employment is in percentage points. * does not pass the stricter Bonferroni test explained below.

- **Does each group gain?** Yes. At 30 months every range is above zero. ST has the largest estimates and the widest ranges, since it is the smallest group.
- **Do groups gain different amounts?** We cannot say. We tested all 24 gaps between groups directly and none is below 5% (smallest p = 0.10). ST's lead over General on 30-month earnings is ₹3,632, with a 95% range from −₹2,849 to ₹10,114. The offer raised enrolment about equally in every group (57.7 to 59.5 points).
- **Do ST's gains come later?** Possibly. Its 18-month effects are unclear, and they rise by 4.8 points and ₹7,870 by 30 months, but neither rise passes the stricter test for making 8 such comparisons.
- **The risk of many tests:** the table holds 16 tests. If there were no real effects and the tests were independent, the chance of at least one false alarm would be 56%. So we also applied the Bonferroni test, which judges each result at 0.05 / 16. 12 of the 16 still pass, including all four 30-month earnings effects.

5. Inference and Recommendations

Question	Answer	Why
Continue the programme?	Yes	The offer clearly raises employment and earnings.
Expand it?	Cannot say yet	The gain of ₹9,076 is clearly above zero (statistically significant) and 13.5% of the not offered group's earnings (a meaningful size). Whether it is worth paying for (policy significance) depends on the cost per place, which is not in the data.
Redesign it?	Look at take-up	38% of offers go unused, and each one dilutes the effect per offer. Find out why people decline.
Target one group?	No	Every group gains, and nothing shows that any group gains more.

- **Assumptions:** the offer was random (the checks in section 1 support this), one applicant's offer does not change another's outcome, and outcomes were measured the same way for both groups.
- **Limits:** synthetic data; no costs; the effect of the training itself and the ITT with covariates not done; earnings period not defined; no adjustment for the lottery being run within states.
- **Future evaluations:** record the cost per place, follow people for longer, and use larger samples for small groups.

Code, notebooks and full numbers: https://github.com/IshuIsAwake/advanced_data_science